

Organizing Julia Research Software Projects

Slides assembled with the assistance of Claude (Anthropic) from material by J. Fuhrmann at <https://j-fu.github.io/marginalia/julia/>.

J. Fuhrmann

2026-06-25

Getting Started

Why Julia for Research Software?

- **Performance:** JIT-compiled, near-C speed without manual optimization
- **Expressiveness:** High-level syntax, close to mathematical notation
- **Package ecosystem:** 10000+ registered packages; best-in-class package manager
- **Reproducibility:** Built-in environment management via **Project.toml** and **Manifest.toml**
- **Interactivity:** REPL, Pluto notebooks, Jupyter support

Julia is evolving — current LTS version: **1.10**, current stable: **1.12**

Installing Julia: `juliaup`

Use `juliaup` — the recommended installer and version manager

Linux / macOS

```
curl -fsSL https://install.julialang.org | sh
```

Windows (PowerShell)

```
winget install julia -s msstore
```

- Installs Julia and keeps it up to date
- Manages multiple Julia versions side by side
- **Avoid** OS package managers (Homebrew, apt, ...) — versions are often outdated

See: <https://julialang.org/downloads>

Managing Julia Versions with `juliaup`

```
juliaup status          # list installed versions  
juliaup add 1.11        # install a specific version  
juliaup default 1.11    # set default version  
juliaup update          # update all installed versions  
julia +1.10 myscript.jl # run with a specific version
```

`juliaup` is analogous to `rustup` (Rust) or `pyenv` (Python)

Visual Studio Code with the Julia extension is the recommended editor

- Integrated REPL, inline output, debugger, profiler
- <https://www.julia-vscode.org>

Other editors: Emacs (`julia-emacs`), Zed, Vim/Neovim

The REPL is Julia's interactive shell — keep it running while you work

```
$ julia
julia>          # normal mode
]              # package manager (Pkg) mode
;              # shell mode
?              # help mode
```

Working with Julia Code

Write Code in Functions, Not Scripts

Running scripts in global scope is a common pitfall:

```
# BAD: MyScript.jl in global scope  
using Plots  
x = 0:0.1:2π           # untyped global variable  
y = sin.(x)            # no JIT optimization possible  
plot(x, y)
```

Problems: No JIT optimization, precompilation overhead on every run

```
# GOOD: wrap in a function  
function main(; n=100)  
    x = range(0, 2π; length=n)  
    plot(x, sin.(x))  
end
```

See: [Julia performance tips — avoid global variables](#)

Work from the REPL with `include()`

Start Julia once and reload code without restarting:

```
$ julia --project=.
```

```
julia> include("scripts/myscript.jl")
```

```
julia> main(n=200)
```

after editing myscript.jl:

```
julia> include("scripts/myscript.jl")    # reload
```

```
julia> main(n=200)
```

With Julia < v1.12, when **redefining a struct or a constant**, restart is needed.

Automatic Reloading with Revise.jl

Revise.jl watches source files and recompiles changes automatically.

One-time setup — add to ~/.julia/config/startup.jl:

```
using Revise
```

Then use `includet` (tracked include) instead of `include`:

```
julia> includet("scripts/myscript.jl")
julia> main()           # uses the current version
# edit myscript.jl ...
julia> main()           # automatically sees changes!
```

For packages loaded via `using`, Revise tracks all included files automatically.

Wrap Scripts in Modules

Avoid name clashes when loading multiple scripts in one session:

```
# scripts/myscript.jl
module MyScript
using Plots

function main(; n=100)
    x = range(0, 2 $\pi$ ; length=n)
    plot(x, sin.(x))
end

end # module
```

```
julia> includet("scripts/myscript.jl")
julia> MyScript.main(n=200)
```

Environments and Reproducibility

What Is a Julia Environment?

An **environment** defines which packages are available for `using` / `import`.

It is described by two files in a directory:

File	Purpose
<code>Project.toml</code>	Direct dependencies + version constraints
<code>Manifest.toml</code>	Full dependency tree with exact versions

Julia resolves `using Package` by searching the active environment(s).

Default location: `~/.julia/environments/v1.x/`

```
julia> using Pkg
```

```
julia> Pkg.add("Plots")    # installs into global env
```

Problem with a shared global environment:

- Package version conflicts between projects
- Hard to share: collaborators must manually figure out what to install
- No reproducibility guarantee
*Use the global environment only for personal development tools (e.g. **Revise**, **BenchmarkTools**, **JuliaFormatter**)*

Local Project Environments

Each project gets its **own** environment:

```
$ cd MyProject  
$ julia --project=.
```

```
julia> using Pkg  
julia> Pkg.add("DifferentialEquations")  
julia> Pkg.add("Plots")
```

This creates `MyProject/Project.toml` and `MyProject/Manifest.toml`.

All package installs stay local to **MyProject** — no conflicts with other projects.

Project.toml: What It Contains

```
name = "MyProject"  
uuid = "8f4d0f85-..."  
version = "0.1.0"  
  
[deps]  
DifferentialEquations = "0c46a032-..."  
Plots = "91a5bcdd-..."  
  
[compat]  
julia = "1.10"  
DifferentialEquations = "7"  
Plots = "1"
```

- Lists direct dependencies with UUIDs
- `[compat]` constrains acceptable versions
- Should be committed to version control

Manifest.toml: Reproducibility

```
julia_version = "1.11.0"  
manifest_format = "2.0"  
  
[[deps.DifferentialEquations]]  
deps = ["BoundaryValueDiffEq", "...]  
git-tree-sha1 = "abc123..."  
version = "7.13.0"  
# ... hundreds more entries ...
```

- Records **exact versions** of every dependency (direct and transitive)
- Enables bit-for-bit reproducible environments
- **Committing it** → anyone can reproduce the exact environment
- **Not committing it** → collaborators get latest compatible versions

Key Package Manager Commands

```
julia> using Pkg
```

```
Pkg.activate(".")           # activate local environment  
Pkg.add("SomePackage")     # add a dependency  
Pkg.rm("SomePackage")      # remove a dependency  
Pkg.update()               # update all packages  
Pkg.status()               # list installed packages  
Pkg.instantiate()          # install all deps from Project.toml  
Pkg.resolve()              # re-resolve and update Manifest.toml
```

Shortcut: press] in the REPL to enter **Pkg mode**:

```
(MyProject) pkg> add Plots  
(MyProject) pkg> status
```

Activating a local environment **does not hide** the global one:

Active environments (search order):

1. `MyProject/` (local) ← project dependencies
 2. `~/.julia/environments/v1.11/` (global) ← dev tools
- Project dependencies go in the **local** environment
 - Development tools (**Revise**, **BenchmarkTools**) stay in the **global** environment
 - Clean separation: tools don't pollute the reproducible project environment

Research Projects vs. Packages

Packages vs. Research Projects

	Package	Research Project
Purpose	Reusable library for others	Produce specific results/paper
Users	External users, many versions	Small team, evolves with the work
Data	No data	Central: raw data, results, figures
Workflow	Stable API, tests, docs	Exploratory, trial-and-error
Sharing	Registered in a registry	Shared via git URL
Notebooks/scripts	Optional	Core deliverables

A research project in Julia corresponds to an **application** in the Pkg glossary.

Challenges of Research Software

Research software tends to be:

- **Exploratory:** requirements change as understanding grows
- **Collaborative:** shared between group members, possibly with different Julia versions
- **Long-lived:** may need to reproduce results years later
- **Mixed:** code + data + notebooks + paper drafts in one place
- **Underdocumented:** under time pressure, tests and docs are often skipped

These challenges make **good organization even more important**, not less.

Why Organize a Research Project Like a Package?

Structuring a research project as a (non-registered) package gives you:

- **Shared environment** (`Project.toml`) — one `Pkg.instantiate()` sets up everything
- **Shared source code** in `src/` — import project utilities with `using MyProject`
- **Unit testing** via `test/runtests.jl` — catch regressions early
- **Documentation** with `Documenter.jl` — explain your methods
- **Version control ready** — standard layout, easy to push to git

This is sustainable research software with minimal overhead.

Structuring the Project

Recommended Project Layout

```
MyProject/
├── Project.toml      ← environment + package definition
├── Manifest.toml    ← (optionally committed) exact versions
├── README.md
├── LICENSE
├── src/
│   └── MyProject.jl ← shared project code (a Julia module)
├── scripts/
│   └── simulation.jl ← scripts that produce results
├── notebooks/
│   └── analysis.jl  ← Pluto notebooks
├── test/
│   └── runtests.jl  ← unit tests
├── docs/            ← Documenter.jl sources
└── papers/          ← manuscripts
```

Scripts: No Activation Needed

Start Julia from the project root — the environment is active for all scripts:

```
$ cd MyProject  
$ julia --project=.
```

Scripts need no explicit activation code:

```
# scripts/simulation.jl  
using MyProject           # shared project code from src/  
using Plots, DifferentialEquations
```

```
function main()  
    # ...  
end
```

```
julia> includet("scripts/simulation.jl")  
julia> main()
```

Pluto Notebooks: Disabling the Built-in Package Manager

Pluto has its own built-in package manager — great for standalone notebooks, but it must be **disabled** to share the project environment.

Add a Pkg cell at the top of the notebook:

```
# Pkg cell – triggers Pluto to hand over package management  
using Pkg  
Pkg.activate(joinpath(@__DIR__, "..")) # activate MyProject environment
```

Advantages of Pluto for research projects:

- Reactive: cells re-run automatically on change
- Notebooks are plain `.jl` files — diff-friendly in git
- HTML/PDF export built in
- `Revise.jl` works normally once the built-in manager is disabled

Sub-packages Inside a Project

Code that grows can become a standalone package inside `packages/`:

```
MyProject/  
└─ packages/  
    └─ MySubPackage/  
        ├── Project.toml  
        └─ src/  
            └─ MySubPackage.jl
```

Add it as a development dependency:

```
pkg> dev packages/MySubPackage
```

Later, `MySubPackage` can be extracted and registered independently — scripts depending on it don't need to change.

To Commit `Manifest.toml` or Not?

Commit it when:

- Strong reproducibility is required (paper submission, archiving)
- Working with a fixed Julia version in the team
- Subpackages in relative paths (Julia < 1.11)

Don't commit it when:

- Team members use different Julia versions
- You want to track that the project works with a range of dependency versions
- Using `[compat]` in `Project.toml` for version management

Starting with Julia 1.11: per-version manifests (`Manifest-v1.11.toml`) make committing more practical for multi-version teams.

DrWatson.jl

[DrWatson.jl](#) is a scientific project assistant that helps with:

- **Project-relative paths** — no more hardcoded absolute paths
- **Standardized file naming** — encode simulation parameters in file names
- **Simulation management** — avoid rerunning finished simulations
- **Tagging** — record git commit hash with saved results

```
pkg> add DrWatson
```

Designed to work with the project-as-package layout described [here](#).

Forget about `joinpath(dirname(@__FILE__), "..", "data")`:

start Julia with: julia --project=.

using DrWatson

`datadir()` *# → MyProject/data/*

`plotsdir()` *# → MyProject/plots/*

`srcdir()` *# → MyProject/src/*

`scriptsdir()` *# → MyProject/scripts/*

`papersdir()` *# → MyProject/papers/*

Subdirectories:

`datadir("sims", "run01")` *# → MyProject/data/sims/run01/*

Works from any script or notebook regardless of where it is in the tree.

Encode simulation parameters in file names automatically:

```
params = Dict{"N" => 100, "dt" => 0.01, "method" => "rk4"}
```

```
savename(params)
```

```
# → "N=100_dt=0.01_method=rk4"
```

```
savename(params; suffix="jld2")
```

```
# → "N=100_dt=0.01_method=rk4.jld2"
```

```
# Save results:
```

```
@tagsave(datadir("sims", savename(params, suffix="jld2")), results)
```

@tagsave adds git commit information to the saved data automatically.

```
function run_simulation(params)
    # expensive computation...
    return results
end
```

Run only if output file does not exist yet:

```
results, path = produce_or_load(run_simulation, params, datadir("sims"))
```

`produce_or_load` checks for an existing output file matching the parameters. If found, it loads the result; otherwise it runs the function and saves.

Ideal for parameter sweeps and long-running simulations.

Hands-On: julia-project-skeleton

A ready-to-use template that implements all recommendations:

<https://github.com/j-fu/julia-project-skeleton>

Download and unpack [julia-project-skeleton-2.0.0.zip](#) from the releases page, then use the path to the unpacked directory as the template argument.

Generate an initial project version using [PkgSkeleton.jl](#):

```
$ julia
julia> using Pkg
julia> Pkg.add("PkgSkeleton")
julia> using PkgSkeleton
julia> PkgSkeleton.generate("MyProject";
                           templates=["julia-project-skeleton-2.0.0"])
```

Setting Up a New Project: Step by Step

Generate the skeleton (see previous slide), then install dependencies:

```
$ cd MyProject  
$ julia --project  
julia> using Pkg; Pkg.instantiate()
```

Then adapt it to your needs:

- Replace demo files with your own scripts and notebooks
- Update `[deps]` in `Project.toml` as needed (`pkg> add ...`)
- Update `authors`, `name`, and `version` in `Project.toml`
- Push to git early — and commit often

Running Code in the Skeleton

The skeleton ships ready-to-run demos:

REPL workflow with live reloading:

```
$ julia --project
```

```
julia> using Revise
```

```
julia> includet("scripts/DemoREPL.jl")
```

```
julia> DemoREPL.main(dim=3)
```

run a CLI script directly:

```
$ julia --project scripts/demo-cli.jl
```

run the demo Pluto notebook (Pluto need not be installed globally):

```
$ julia --project etc/runpluto.jl notebooks/demo-notebook.jl
```

Onboarding a Collaborator

Clone the repository

```
$ git clone https://github.com/you/MyProject
```

```
$ cd MyProject
```

Instantiate – downloads and precompiles all dependencies

```
$ julia --project
```

```
julia> using Pkg; Pkg.instantiate()
```

Run a CLI script

```
$ julia --project scripts/demo-cli.jl
```

or use the REPL with live reloading

```
julia> using Revise
```

```
julia> includet("scripts/DemoREPL.jl")
```

```
julia> DemoREPL.main(dim=3)
```

No manual package installation, no version hunting — just `instantiate`

Summary

Summary: Key Recommendations

1. Install Julia via **juliaup** — easy version management
2. Write code in functions — enable JIT optimization
3. Stay in the REPL — use `include()` / `Revise.jl` instead of re-running
4. Use a local environment per project — `julia --project=.`
5. Commit **Project.toml** always; commit **Manifest.toml** when strong reproducibility is needed
6. Structure the project as a package — shared env, shared `src/`, tests, docs
7. Use `DrWatson.jl` — path helpers, reproducible file naming, simulation management
8. Use **julia-project-skeleton** — get the layout right from the start

By following these practices, a Julia research project achieves all five criteria for sustainable research software (A. Zeller):

Criterion	How
Have a repo	git from day one
Anyone can build	<code>Pkg.instantiate()</code>
Have tests	<code>test/runtests.jl</code>
Open for extensions	modular <code>src/</code> + sub-packages
Have examples	<code>scripts/</code> + <code>notebooks/</code>

Julia's package management infrastructure makes this largely automatic.

Outlook: Newer Developments (Julia $\geq$ 1.11)

Topics for a follow-up talk:

- **[sources] in Project.toml** — declare in-tree sub-packages by relative path; sub-package discovery no longer requires committing **Manifest.toml**
- **Workspaces** (Pkg) — manage a set of related packages with a shared environment; natural fit for multi-package research projects
- **@main** entry point — designate a **main(args)** function as the CLI entry point; cleaner scripts without boilerplate
- **LocalRegistry.jl** — host a private registry for your team or institution; share registered packages without publishing to General
- **Sub-packages as registered packages** — register while keeping code in the project tree; switch from `pkg> dev` to `pkg> add` when ready

References and Further Reading

- Julia documentation: <https://docs.julialang.org>
- Pkg documentation (environments, `Project.toml`, `Manifest.toml`): <https://pkgdocs.julialang.org>
- Modern Julia Workflows (G. Dalle, J. Smit, A. Hill, 2024): <https://modernjuliaworkflows.github.io>
- DrWatson.jl: <https://github.com/JuliaDynamics/DrWatson.jl>
- PkgTemplates.jl: <https://github.com/JuliaCI/PkgTemplates.jl>
- julia-project-skeleton: <https://github.com/j-fu/julia-project-skeleton>
- Basic Workflow: <https://j-fu.github.io/marginalia/julia/basic-workflow>
- Project Workflow: <https://j-fu.github.io/marginalia/julia/project-workflow>
- Project Howto: <https://j-fu.github.io/marginalia/julia/project-howto>
- Markdown source:
<https://raw.githubusercontent.com/j-fu/marginalia/refs/heads/main/julia/julia-project-org.md>